INFORMATION SYSTEMS AND DATABASES

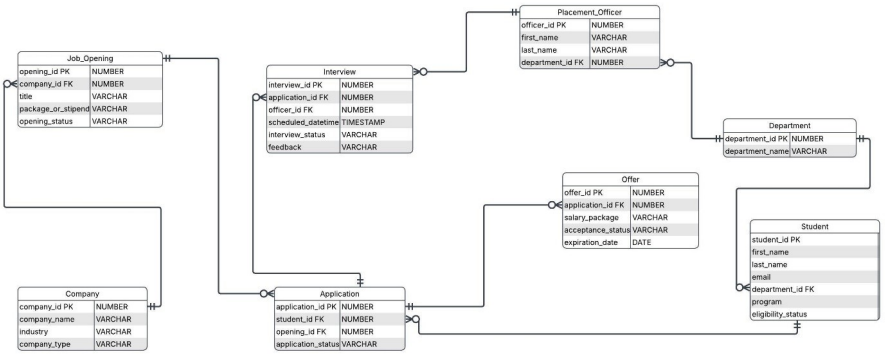
Portfolio Element 2 — University Placement Management System

Author: Rachelle Phipps
Platform: Oracle APEX 26.1

1. Assumptions / Business Rules

- Each student belongs to one specific academic department. This allows us to track performance by department in Query 1.
- A department can have multiple students and placement officers, which helps track officer workloads in Query 6.
- Each company can post multiple job openings, but each opening belongs to only one company. This lets us link vacancies to company details for Queries 4 and 8.
- Students are allowed to submit multiple applications, which the database tracks individually from submission to offer stage.
- An application can have multiple interview stages, with each interview managed by a single placement officer for Query 6 reporting.
- Applications do not automatically receive an offer. Offers are tracked by statuses such as "Accepted" or "Pending" to calculate success rates.
- Pending offers include expiry dates so placement officers can easily identify urgent follow-ups for Query 5.
- Job openings must store salary data, deadlines and statuses (such as Open or Closed) so the system can filter for active roles and analyse financial value.

2. ER Diagram



Clarifications and Design Improvements

Normalisation Decisions

1. Student names were removed from the Application table because student information is already stored in the Student table. This reduces duplicate data and follows database normalisation principles.
2. Placement officers were linked to departments using a department ID instead of storing department names as plain text. This improves consistency and reduces repeated data throughout the database.
3. Company information is stored separately in the Company table, while job openings only store the company ID. This avoids repeating company details for every job opening.

4. Student applications are stored in a separate Application table instead of inside the Student or Job Opening tables. This allows one student to apply for multiple roles without duplicating student or job data.
5. Interview information is stored in a separate Interview table so multiple interviews can be linked to one application without repeating application details.

Other Design Improvements

1. CHECK constraints were added to fields such as application status, interview status and interview mode to ensure only valid values can be entered into the system.
2. UNIQUE constraints were added to student and placement officer email addresses to prevent duplicate accounts.
3. A feedback field was added to the Interview table so placement officers can store interview comments and outcomes directly in the database.
4. Identity columns were used for primary keys so IDs are automatically generated when new records are inserted.
5. Status fields such as opening status, application status and acceptance status were included to support reporting and tracking throughout the placement process.
6. Primary keys and foreign keys were added to all tables to maintain relationships and ensure referential integrity.
7. The database was designed so one student can apply for multiple job openings, and one application can have multiple interview rounds, reflecting the real placement process used by universities.

3. Entity Specification Forms

The following forms specify every entity shown on the ER diagram. Each form lists the attributes, data types, key status, validation rules and example values.

ENTITY: DEPARTMENT

Stores details of university departments.

Attribute	Data type & width	Status (pk/fk/nn)	Validation	Example / notes
department_id	NUMBER	pk	Auto generated	1
department_name	VARCHAR2(100)	nn	Cannot be empty	Computer Science

ENTITY: STUDENT

Stores student details.

Attribute	Data type & width	Status (pk/fk/nn)	Validation	Example / notes
student_id	NUMBER	pk	Auto generated	1
first_name	VARCHAR2(100)	nn	Cannot be empty	John
last_name	VARCHAR2(100)	nn	Cannot be empty	Smith
email	VARCHAR2(100)	nn	Must be unique	john@email.com
phone	VARCHAR2(20)			+44 123456789
department_id	NUMBER	fk	Must exist in Department	1
program	VARCHAR2(100)			BSc Computer Science
graduation_year	NUMBER(4)		4 digit year	2026
resume_url	VARCHAR2(500)			resume.pdf
eligibility_status	VARCHAR2(50)		Eligible / Ineligible / Blacklisted	Eligible

ENTITY: COMPANY

Stores company details.

Attribute	Data type & width	Status (pk/fk/nn)	Validation	Example / notes
company_id	NUMBER	pk	Auto generated	1
company_name	VARCHAR2(255)	nn	Cannot be empty	TechGlobal
industry	VARCHAR2(100)			Technology
contact_person	VARCHAR2(100)			Sarah Khan
contact_email	VARCHAR2(100)			contact@techglobal.com
contact_phone	VARCHAR2(20)			+44 789654321
company_type	VARCHAR2(50)			MNC

ENTITY: JOB_OPENING

Stores job openings.

Attribute	Data type & width	Status (pk/fk/nn)	Validation	Example / notes
opening_id	NUMBER	pk	Auto generated	1
company_id	NUMBER	fk	Must exist in Company	1

Attribute	Data type & width	Status (pk/fk/nn)	Validation	Example / notes
title	VARCHAR2(200)	nn	Cannot be empty	Software Engineer
role_type	VARCHAR2(100)			Full-time
location	VARCHAR2(100)			London
package_or_stipend	VARCHAR2(100)			£45k
required_skills	VARCHAR2(500)			SQL, Python
application_deadline	DATE		Valid date	20-JUN-2026
opening_status	VARCHAR2(50)		Open / Closed / On Hold / Cancelled	Open

ENTITY: APPLICATION

Stores student applications.

Attribute	Data type & width	Status (pk/fk/nn)	Validation	Example / notes
application_id	NUMBER	pk	Auto generated	1
student_id	NUMBER	fk	Must exist in Student	1
opening_id	NUMBER	fk	Must exist in Job_Opening	1
applied_date	DATE		Defaults to current date	01-MAY-2026
application_status	VARCHAR2(50)		Applied / Offered / Rejected / In-Progress	Offered

ENTITY: PLACEMENT_OFFICER

Stores placement officer details.

Attribute	Data type & width	Status (pk/fk/nn)	Validation	Example / notes
officer_id	NUMBER	pk	Auto generated	1
first_name	VARCHAR2(100)	nn	Cannot be empty	Jane
last_name	VARCHAR2(100)	nn	Cannot be empty	Doe
email	VARCHAR2(100)		Must be unique	j.doe@uni.ac.uk
phone	VARCHAR2(20)			+44 777777777
department_id	NUMBER	fk	Must exist in Department	1

ENTITY: INTERVIEW

Stores interview details.

Attribute	Data type & width	Status (pk/fk/nn)	Validation	Example / notes
interview_id	NUMBER	pk	Auto generated	1
application_id	NUMBER	fk	Must exist in Application	1
officer_id	NUMBER	fk	Must exist in Placement_Officer	1
scheduled_datetime	TIMESTAMP	nn	Must contain date and time	26-MAY-2026 11:00
interview_mode	VARCHAR2(50)		Online / In-Person / Telephone	Online
round_name	VARCHAR2(100)			Technical
interview_status	VARCHAR2(50)		Scheduled / Completed /	Completed

Attribute	Data type & width	Status (pk/fk/nn)	Validation	Example / notes
			Rejected	
feedback	VARCHAR2(4000)			Good technical skills

ENTITY: OFFER

Stores offer details.

Attribute	Data type & width	Status (pk/fk/nn)	Validation	Example / notes
offer_id	NUMBER	pk	Auto generated	1
application_id	NUMBER	fk	Must exist in Application	1
salary_package	VARCHAR2(100)			£45,000
joining_date	DATE		Valid date	15-JUL-2026
offer_letter_url	VARCHAR2(500)			offer1.pdf
acceptance_status	VARCHAR2(50)		Accepted / Pending / Rejected / Declined	Accepted
expiration_date	DATE		Valid date	30-JUN-2026

4. SQL Table Creation Scripts

Each script below matches its Entity Specification Form exactly, including all columns, keys and constraints.

1. Department

```
CREATE TABLE Department (
    department_id   NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    department_name VARCHAR2(100) NOT NULL
);
```

2. Company

```
CREATE TABLE Company (
    company_id     NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    company_name   VARCHAR2(255) NOT NULL,
    industry       VARCHAR2(100),
    contact_person VARCHAR2(100),
    contact_email  VARCHAR2(100),
    contact_phone  VARCHAR2(20),
    company_type   VARCHAR2(50)
);
```

3. Student

```
CREATE TABLE Student (
    student_id        NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    first_name        VARCHAR2(100) NOT NULL,
    last_name         VARCHAR2(100) NOT NULL,
    email             VARCHAR2(100) UNIQUE,
    phone             VARCHAR2(20),
    department_id     NUMBER,
    program           VARCHAR2(100),
    graduation_year    NUMBER(4),
    resume_url        VARCHAR2(500),
    eligibility_status VARCHAR2(50),
    CONSTRAINT fk_student_dept FOREIGN KEY (department_id) REFERENCES Department(department_id),
    CONSTRAINT chk_eligibility CHECK (eligibility_status IN ('Eligible','Ineligible','Blacklisted'))
);
```

4. Placement_Officer

```
CREATE TABLE Placement_Officer (
    officer_id    NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    first_name    VARCHAR2(100) NOT NULL,
    last_name     VARCHAR2(100) NOT NULL,
    email         VARCHAR2(100) UNIQUE,
    phone         VARCHAR2(20),
    department_id NUMBER,
    CONSTRAINT fk_officer_dept FOREIGN KEY (department_id) REFERENCES Department(department_id)
);
```

5. Job_Opening

```
CREATE TABLE Job_Opening (
    opening_id         NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    company_id         NUMBER,
```

```
    title              VARCHAR2(200) NOT NULL,
    role_type          VARCHAR2(100),
    location           VARCHAR2(100),
    package_or_stipend   VARCHAR2(100),
    required_skills      VARCHAR2(500),
    application_deadline DATE,
    opening_status       VARCHAR2(50),
    CONSTRAINT fk_job_company FOREIGN KEY (company_id) REFERENCES Company(company_id),
    CONSTRAINT chk_op_status CHECK (opening_status IN ('Open','Closed','On Hold','Cancelled'))
);
```

6. Application

```
CREATE TABLE Application (
    application_id     NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    student_id         NUMBER,
    opening_id         NUMBER,
    applied_date       DATE DEFAULT SYSDATE,
    application_status VARCHAR2(50) DEFAULT 'Applied',
    CONSTRAINT fk_app_student FOREIGN KEY (student_id) REFERENCES Student(student_id),
    CONSTRAINT fk_app_opening FOREIGN KEY (opening_id) REFERENCES Job_Opening(opening_id),
    CONSTRAINT chk_app_status CHECK (application_status IN ('Applied','Offered','Rejected','In-Progress'))
);
```

7. Interview

```
CREATE TABLE Interview (
    interview_id       NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    application_id     NUMBER,
    officer_id         NUMBER,
    scheduled_datetime TIMESTAMP NOT NULL,
    interview_mode     VARCHAR2(50),
    round_name         VARCHAR2(100),
    interview_status   VARCHAR2(50),
    feedback           VARCHAR2(4000),
    CONSTRAINT fk_int_app     FOREIGN KEY (application_id) REFERENCES Application(application_id),
    CONSTRAINT fk_int_officer FOREIGN KEY (officer_id) REFERENCES Placement_Officer(officer_id),
    CONSTRAINT chk_int_mode   CHECK (interview_mode IN ('Online','In-Person','Telephone')),
    CONSTRAINT chk_int_status CHECK (interview_status IN ('Scheduled','Completed','Rejected'))
);
```

8. Offer

```
CREATE TABLE Offer (
    offer_id         NUMBER GENERATED BY DEFAULT AS IDENTITY PRIMARY KEY,
    application_id   NUMBER,
    salary_package   VARCHAR2(100),
    joining_date     DATE,
    offer_letter_url VARCHAR2(500),
    acceptance_status VARCHAR2(50) DEFAULT 'Pending',
    expiration_date  DATE,
    CONSTRAINT fk_offer_app    FOREIGN KEY (application_id) REFERENCES Application(application_id),
    CONSTRAINT chk_offer_status CHECK (acceptance_status IN ('Accepted','Pending','Rejected','Declined'))
);
```

5. Sample Data

Below is sample data for each table. The full data sets provide sufficient breadth and volume to support the complex testing required by the reports in Section 6.

Department Table

Full dataset of 8 records. All 8 departments are included so that every department_id referenced by students and officers resolves to a valid row.

```
INSERT INTO department (department_id, department_name) VALUES (1, 'Computer Science');
INSERT INTO department (department_id, department_name) VALUES (2, 'Business Analytics');
INSERT INTO department (department_id, department_name) VALUES (3, 'Mechanical Engineering');
INSERT INTO department (department_id, department_name) VALUES (4, 'Digital Media');
INSERT INTO department (department_id, department_name) VALUES (5, 'Psychology');
INSERT INTO department (department_id, department_name) VALUES (6, 'Applied Economics');
INSERT INTO department (department_id, department_name) VALUES (7, 'International Law');
INSERT INTO department (department_id, department_name) VALUES (8, 'Public Health');
```

Placement Officer Table

Full dataset of 6 records.

```
INSERT INTO placement_officer (officer_id, first_name, last_name, email, phone, department_id)
VALUES (1, 'Sarah', 'Cook', 'officer.1@university.ac.uk', '+44 7940 974316', 2);
INSERT INTO placement_officer (officer_id, first_name, last_name, email, phone, department_id)
VALUES (2, 'Andrew', 'Hill', 'officer.2@university.ac.uk', '+44 7998 325064', 1);
INSERT INTO placement_officer (officer_id, first_name, last_name, email, phone, department_id)
VALUES (3, 'Sandra', 'Davis', 'officer.3@university.ac.uk', '+44 7813 971417', 7);
INSERT INTO placement_officer (officer_id, first_name, last_name, email, phone, department_id)
VALUES (4, 'Susan', 'Knight', 'officer.4@university.ac.uk', '+44 7956 404757', 2);
```

Company Table

Full dataset of 30 records.

```
INSERT INTO company (company_id, company_name, industry, contact_person, contact_email, contact_phone,
company_type)
VALUES (1, 'Aether Corp', 'Education', 'Sarah Williams', 'recruitment@aethercorp.com', '+44 7896 460763',
'Startup');
INSERT INTO company (company_id, company_name, industry, contact_person, contact_email, contact_phone,
company_type)
VALUES (2, 'Blue Sky Dynamics', 'Finance', 'Emily Green', 'recruitment@blueskydynamics.com', '+44 7979
225506', 'SME');
INSERT INTO company (company_id, company_name, industry, contact_person, contact_email, contact_phone,
company_type)
VALUES (3, 'Cipher Security', 'Consulting', 'Robert Evans', 'recruitment@ciphersecurity.com', '+44 7785 492747',
'SME');
INSERT INTO company (company_id, company_name, industry, contact_person, contact_email, contact_phone,
company_type)
VALUES (4, 'Data Stream Ltd', 'Retail', 'Steven Watson', 'recruitment@datastreamltd.com', '+44 7880 905809',
'MNC');
```

Student Table

Full dataset of 100 records.

```
INSERT INTO student (student_id, first_name, last_name, email, phone, department_id, program,
graduation_year, resume_url, eligibility_status)
VALUES (1, 'Brian', 'Davies', 'brian.davies1@email.com', '+44 7958 534697', 7, 'PhD Law', 2026,
'http://uni.ac.uk/cv/davies_1.pdf', 'Eligible');
```

```
INSERT INTO student (student_id, first_name, last_name, email, phone, department_id, program,
graduation_year, resume_url, eligibility_status)
VALUES (2, 'Kimberly', 'Foster', 'kimberly.foster2@email.com', '+44 7872 892195', 1, 'MSc Artificial Intelligence',
2026, 'http://uni.ac.uk/cv/foster_2.pdf', 'Eligible');
INSERT INTO student (student_id, first_name, last_name, email, phone, department_id, program,
graduation_year, resume_url, eligibility_status)
VALUES (3, 'Ashley', 'Hall', 'ashley.hall3@email.com', '+44 7711 221474', 2, 'PhD Business Administration', 2026,
'http://uni.ac.uk/cv/hall_3.pdf', 'Eligible');
INSERT INTO student (student_id, first_name, last_name, email, phone, department_id, program,
graduation_year, resume_url, eligibility_status)
VALUES (4, 'Carol', 'Watson', 'carol.watson4@email.com', '+44 7982 560444', 6, 'MSc Financial Economics', 2026,
'http://uni.ac.uk/cv/watson_4.pdf', 'Eligible');
```

Job Opening Table

Full dataset of 50 records.

```
INSERT INTO job_opening (opening_id, company_id, title, role_type, location, package_or_stipend,
required_skills, application_deadline, opening_status)
VALUES (1, 22, 'Software Engineer', 'Full-time', 'London', '£63000', 'Python, Java, SQL, Git', to_date('19-
JUN-2026','dd-mon-yyyy'), 'Open');
INSERT INTO job_opening (opening_id, company_id, title, role_type, location, package_or_stipend,
required_skills, application_deadline, opening_status)
VALUES (2, 7, 'Business Analyst', 'Full-time', 'Manchester', '£66000', 'SQL, Excel, Tableau, Power BI', to_date('12-
JUL-2026','dd-mon-yyyy'), 'Closed');
INSERT INTO job_opening (opening_id, company_id, title, role_type, location, package_or_stipend,
required_skills, application_deadline, opening_status)
VALUES (3, 7, 'Business Analyst', 'Internship', 'Birmingham', '£36000', 'SQL, Excel, Tableau, Power BI', to_date('25-
JUN-2026','dd-mon-yyyy'), 'Closed');
INSERT INTO job_opening (opening_id, company_id, title, role_type, location, package_or_stipend,
required_skills, application_deadline, opening_status)
VALUES (4, 16, 'Strategy Consultant', 'Internship', 'London', '£84000', 'Data Modeling, SWOT Analysis, Market
Research', to_date('15-JUL-2026','dd-mon-yyyy'), 'Open');
```

Application Table

Full dataset of 60 records.

```
INSERT INTO application (application_id, student_id, opening_id, applied_date, application_status)
VALUES (1, 1, 27, to_date('15-MAY-2026','dd-mon-yyyy'), 'Offered');
INSERT INTO application (application_id, student_id, opening_id, applied_date, application_status)
VALUES (2, 60, 26, to_date('19-APR-2026','dd-mon-yyyy'), 'Offered');
INSERT INTO application (application_id, student_id, opening_id, applied_date, application_status)
VALUES (3, 41, 26, to_date('24-APR-2026','dd-mon-yyyy'), 'Applied');
INSERT INTO application (application_id, student_id, opening_id, applied_date, application_status)
VALUES (4, 67, 45, to_date('23-APR-2026','dd-mon-yyyy'), 'Rejected');
```

Interview Table

Full dataset of 55 records.

```
INSERT INTO interview (interview_id, application_id, officer_id, scheduled_datetime, interview_mode,
round_name, interview_status, feedback)
VALUES (1, 36, 4, to_timestamp('26-MAY-2026 11:04:12','dd-mon-yyyy hh24:mi:ss'), 'In-Person', 'HR', 'Completed',
'Articulate speaker with a clear vision for their career path.');
```

Offer Table

Full dataset of 27 records.

```
INSERT INTO offer (offer_id, application_id, salary_package, joining_date, offer_letter_url, acceptance_status, expiration_date)
VALUES (1, 53, '£62000', to_date('15-JUL-2026','dd-mon-yyyy'), 'http://portal.com/offers/off1.pdf', 'Pending', to_date('30-JUN-2026','dd-mon-yyyy'));
INSERT INTO offer (offer_id, application_id, salary_package, joining_date, offer_letter_url, acceptance_status, expiration_date)
VALUES (2, 42, '£35000', to_date('15-JUL-2026','dd-mon-yyyy'), 'http://portal.com/offers/off2.pdf', 'Accepted', to_date('30-JUN-2026','dd-mon-yyyy'));
INSERT INTO offer (offer_id, application_id, salary_package, joining_date, offer_letter_url, acceptance_status, expiration_date)
VALUES (3, 13, '£72000', to_date('15-JUL-2026','dd-mon-yyyy'), 'http://portal.com/offers/off3.pdf', 'Accepted', to_date('30-JUN-2026','dd-mon-yyyy'));
INSERT INTO offer (offer_id, application_id, salary_package, joining_date, offer_letter_url, acceptance_status, expiration_date)
VALUES (4, 30, '£55000', to_date('15-JUL-2026','dd-mon-yyyy'), 'http://portal.com/offers/off4.pdf', 'Pending', to_date('30-JUN-2026','dd-mon-yyyy'));
```

Annotation of Test Data

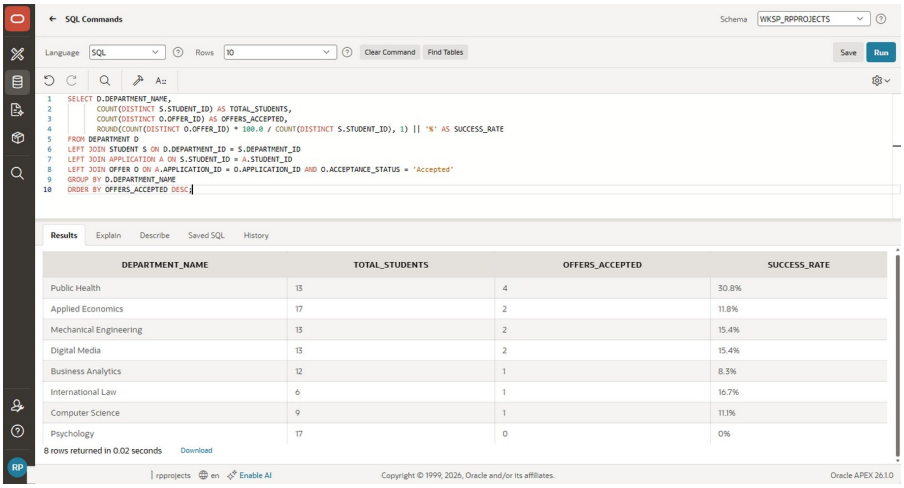
The department records allow the database to test departmental grouping and comparison. Each student record is linked to a department, making it possible to measure departmental placement success. The company and job opening records allow the system to test available roles and employer activity. The application record connects the student to the job opening, which enables application tracking. The interview table tests the interview schedule, officer management and feedback. The offer record allows the database to test accepted offers, including salary information and offer expiry dates. Combined, this data provides enough coverage to test all required reports: placement success by department, student application tracking, interview schedules, officer workload, pending offers, open opportunities, application density and highest-paying roles. The full data set comprises 8 departments, 30 companies, 6 placement officers, 100 students, 50 job openings, 60 applications, 55 interviews and 27 offers, all loaded with referential integrity intact.

6. Reporting Queries

Query 1 — Placement Success Report by Department

- Provides an overview of placement performance across different university departments.

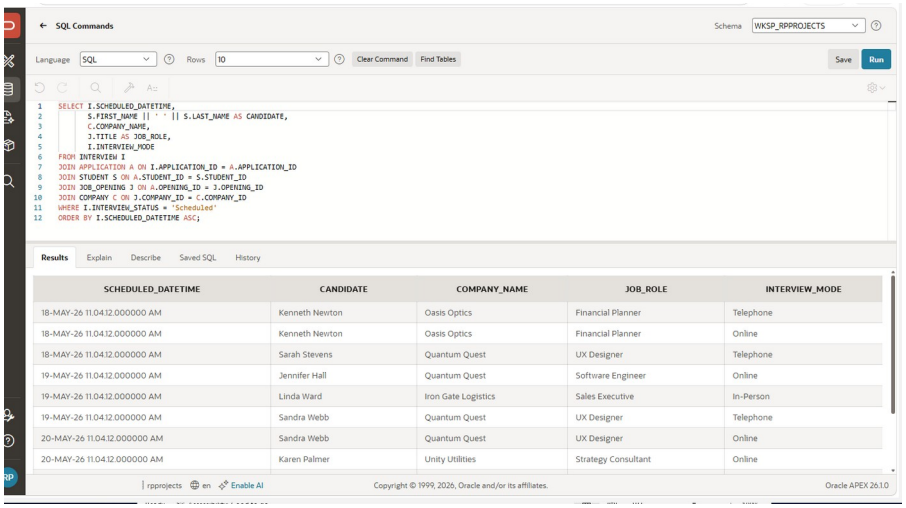
```
SELECT D.DEPARTMENT_NAME,
    COUNT(DISTINCT S.STUDENT_ID) AS TOTAL_STUDENTS,
    COUNT(DISTINCT O.OFFER_ID)   AS OFFERS_ACCEPTED,
    ROUND(COUNT(DISTINCT O.OFFER_ID) * 100.0 /
        NULLIF(COUNT(DISTINCT S.STUDENT_ID), 0), 1) || '%' AS SUCCESS_RATE
FROM DEPARTMENT D
LEFT JOIN STUDENT S     ON D.DEPARTMENT_ID = S.DEPARTMENT_ID
LEFT JOIN APPLICATION A ON S.STUDENT_ID = A.STUDENT_ID
LEFT JOIN OFFER O       ON A.APPLICATION_ID = O.APPLICATION_ID AND O.ACCEPTANCE_STATUS = 'Accepted'
GROUP BY D.DEPARTMENT_NAME
ORDER BY OFFERS_ACCEPTED DESC;
```



Query 2 — Upcoming Interview Schedule

- Gathers a list of all upcoming interviews with details on the student, company and job role.

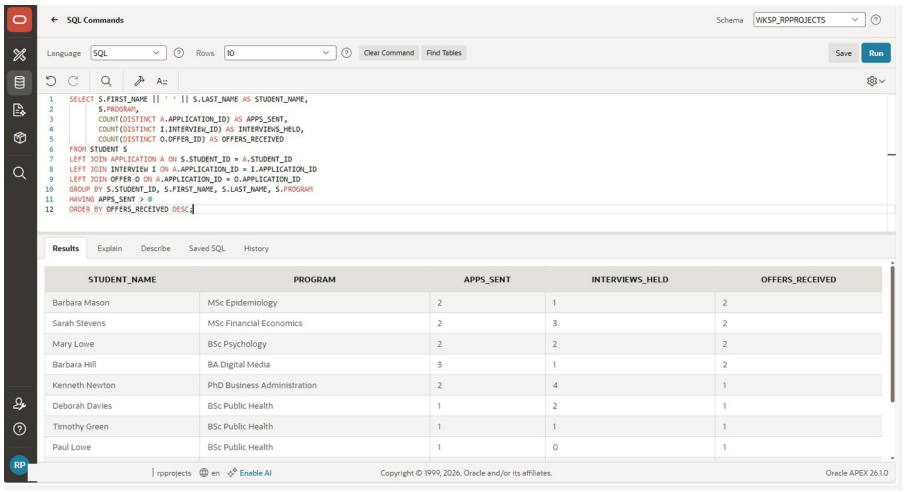
```
SELECT I.SCHEDULED_DATETIME,
    S.FIRST_NAME || ' ' || S.LAST_NAME AS CANDIDATE,
    C.COMPANY_NAME,
    J.TITLE AS JOB_ROLE,
    I.INTERVIEW_MODE
FROM INTERVIEW I
JOIN APPLICATION A ON I.APPLICATION_ID = A.APPLICATION_ID
JOIN STUDENT S     ON A.STUDENT_ID = S.STUDENT_ID
JOIN JOB_OPENING J ON A.OPENING_ID = J.OPENING_ID
JOIN COMPANY C     ON J.COMPANY_ID = C.COMPANY_ID
WHERE I.INTERVIEW_STATUS = 'Scheduled'
ORDER BY I.SCHEDULED_DATETIME ASC;
```



Query 3 — Student Application Tracker

- Tracks the progress of active students from initial application to final offer.

```
SELECT S.FIRST_NAME || ' ' || S.LAST_NAME AS STUDENT_NAME,
    S.PROGRAM,
    COUNT(DISTINCT A.APPLICATION_ID) AS APPS_SENT,
    COUNT(DISTINCT I.INTERVIEW_ID)   AS INTERVIEWS_HELD,
    COUNT(DISTINCT O.OFFER_ID)       AS OFFERS_RECEIVED
FROM STUDENT S
LEFT JOIN APPLICATION A ON S.STUDENT_ID = A.STUDENT_ID
LEFT JOIN INTERVIEW I   ON A.APPLICATION_ID = I.APPLICATION_ID
LEFT JOIN OFFER O       ON A.APPLICATION_ID = O.APPLICATION_ID
GROUP BY S.STUDENT_ID, S.FIRST_NAME, S.LAST_NAME, S.PROGRAM
HAVING COUNT(DISTINCT A.APPLICATION_ID) > 0
ORDER BY OFFERS_RECEIVED DESC;
```

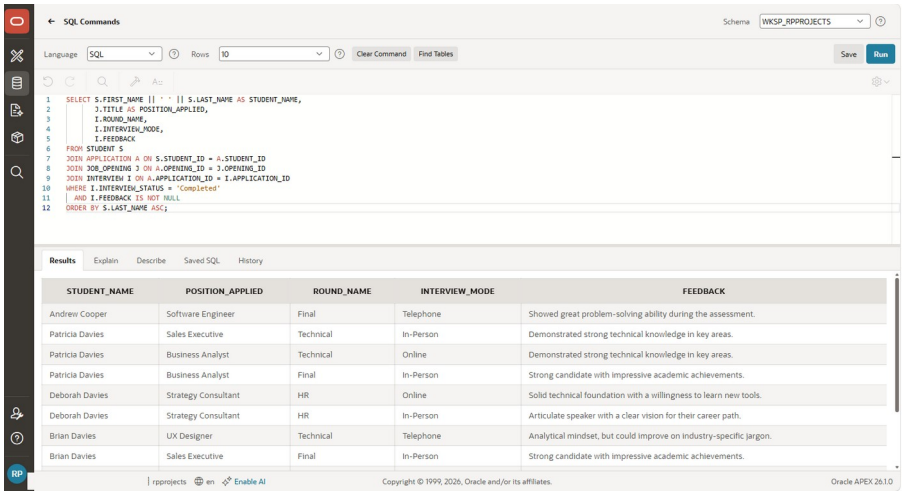


Query 4 — Open Opportunities by Value

- Lists all active job openings with their offer value and application deadline, ranked by package.

```
SELECT c.company_name       AS "Organization",
    c.industry              AS "Sector",
    j.title                 AS "Role",
    j.package_or_stipend    AS "Offer_Value",
    j.application_deadline AS "Deadline"
FROM Job_Opening j
JOIN Company c ON j.company_id = c.company_id
```

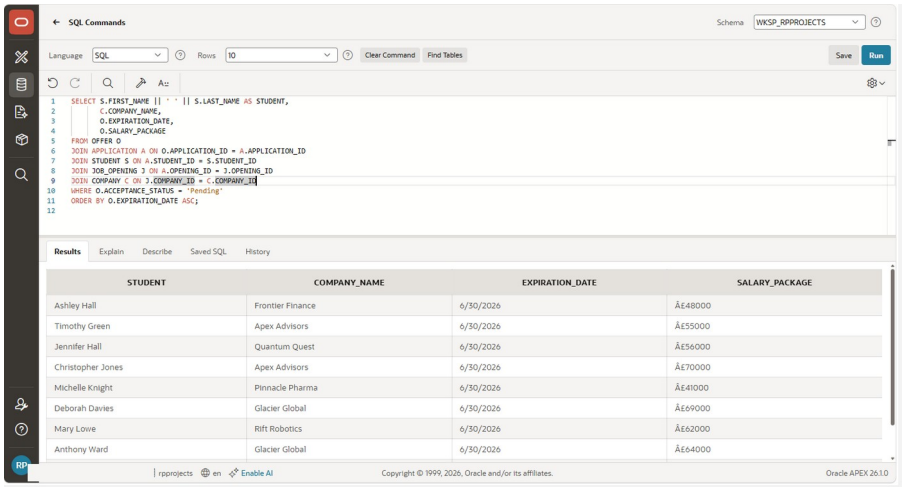
```
WHERE j.opening_status = 'Open'
ORDER BY j.package_or_stipend DESC;
```



Query 5 — Pending Offers Follow-up List

- Identifies students with active offers that are close to expiring.

```
SELECT S.FIRST_NAME || ' ' || S.LAST_NAME AS STUDENT,
    C.COMPANY_NAME,
    O.EXPIRATION_DATE,
    O.SALARY_PACKAGE
FROM OFFER O
JOIN APPLICATION A ON O.APPLICATION_ID = A.APPLICATION_ID
JOIN STUDENT S     ON A.STUDENT_ID = S.STUDENT_ID
JOIN JOB_OPENING J ON A.OPENING_ID = J.OPENING_ID
JOIN COMPANY C     ON J.COMPANY_ID = C.COMPANY_ID
WHERE O.ACCEPTANCE_STATUS = 'Pending'
ORDER BY O.EXPIRATION_DATE ASC;
```

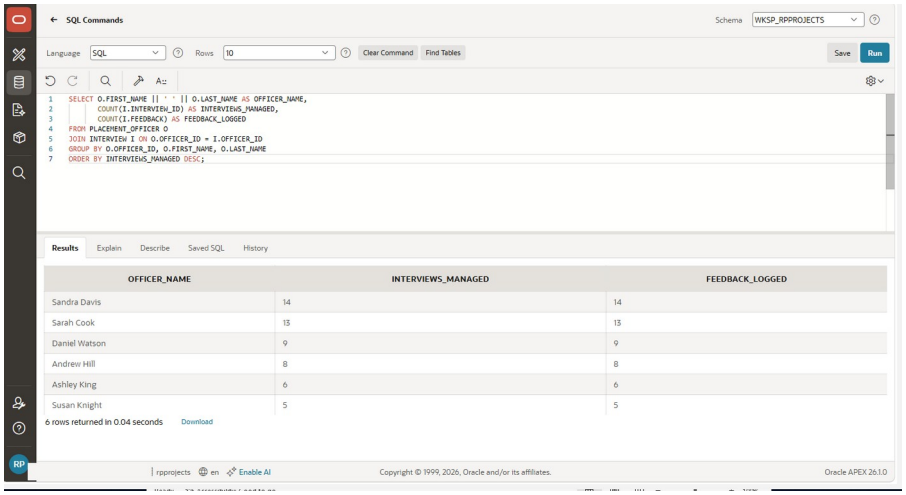


Query 6 — Officer Workload Summary

- Tracks which officers are conducting the most interviews.

```
SELECT O.FIRST_NAME || ' ' || O.LAST_NAME AS OFFICER_NAME,
    COUNT(I.INTERVIEW_ID) AS INTERVIEWS_MANAGED,
    COUNT(I.FEEDBACK)     AS FEEDBACK_LOGGED
FROM PLACEMENT_OFFICER O
```

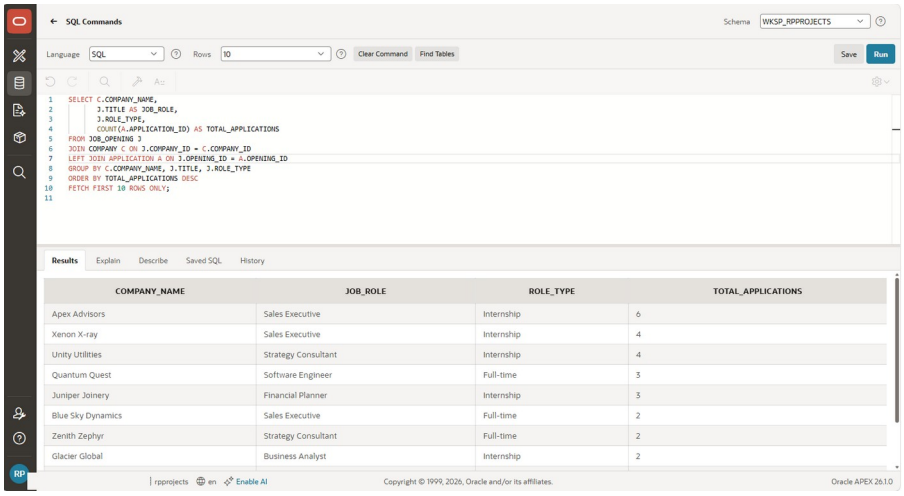
```
JOIN INTERVIEW I ON O.OFFICER_ID = I.OFFICER_ID
GROUP BY O.OFFICER_ID, O.FIRST_NAME, O.LAST_NAME
ORDER BY INTERVIEWS_MANAGED DESC;
```



Query 7 — Application Density

- Determines the job roles and companies receiving the greatest level of student interest. By calculating applications per opening, the placement office can decide which categories are oversubscribed and where additional opportunities are required.

```
SELECT C.COMPANY_NAME,
    J.TITLE AS JOB_ROLE,
    J.ROLE_TYPE,
    COUNT(A.APPLICATION_ID) AS TOTAL_APPLICATIONS
FROM JOB_OPENING J
JOIN COMPANY C        ON J.COMPANY_ID = C.COMPANY_ID
LEFT JOIN APPLICATION A ON J.OPENING_ID = A.OPENING_ID
GROUP BY C.COMPANY_NAME, J.TITLE, J.ROLE_TYPE
ORDER BY TOTAL_APPLICATIONS DESC
FETCH FIRST 10 ROWS ONLY;
```

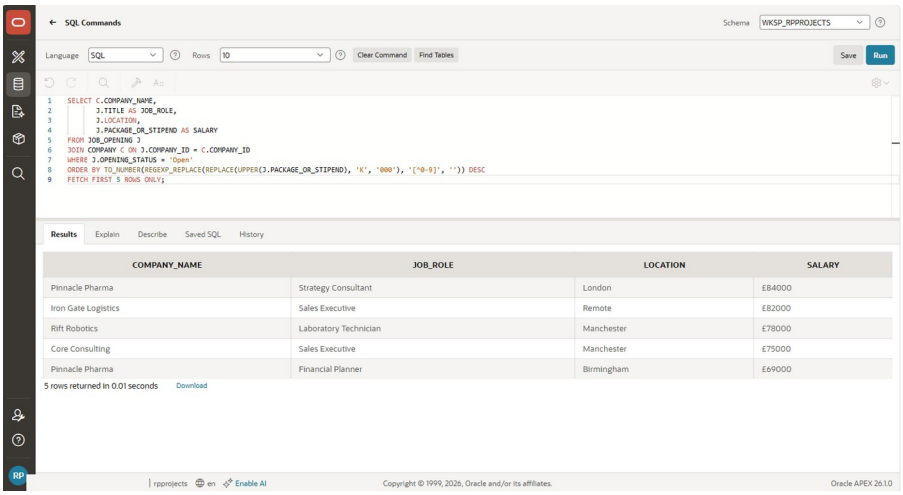


Query 8 — Top 5 Highest Paying Job Roles

- Highlights the highest-paying active job openings available to students.

```
SELECT C.COMPANY_NAME,
    J.TITLE AS JOB_ROLE,
```

```
    J.LOCATION,
    J.PACKAGE_OR_STIPEND AS SALARY
FROM JOB_OPENING J
JOIN COMPANY C ON J.COMPANY_ID = C.COMPANY_ID
WHERE J.OPENING_STATUS = 'Open'
ORDER BY TO_NUMBER(REGEXP_REPLACE(REPLACE(UPPER(J.PACKAGE_OR_STIPEND), 'K', '000'), '[^0-9]', ''))
DESC
FETCH FIRST 5 ROWS ONLY;
```



7. View: Eligible_Students_View

This view was established to facilitate access to students who qualify for placements and internships. It improves convenience and security by enabling placement officers to quickly access only qualified candidates without continually specifying filtering criteria.

Advantages of the View

- Makes it easier to repeat requests for eligible students.
- Enables consistent reporting.
- Restricts access to non-essential student data.
- Assists placement officers in speedy short-listing of candidates.
- Reduces unnecessary filtering conditions in SQL.

SQL Used to Create the View

```
CREATE VIEW Eligible_Students_View AS
SELECT student_id,
    first_name,
    last_name,
    email,
    program,
    graduation_year
FROM Student
WHERE eligibility_status = 'Eligible';
```